

Predicting Running Pace Tier and Average Speed from Strava Wearable-Tracker Data: A Supervised Learning Approach Using kNN, CART, and Linear Regression

Your Names

2026-05-11

Abstract

Chapter 1: Introduction

1.1 Background of the Study

Wearable fitness trackers and GPS-based activity platforms such as **Strava**, **Garmin Connect**, **Nike Run Club**, and **Apple Fitness** have transformed how individuals record and reflect on athletic activity. Each session captures dozens of telemetry features — distance, pace, elevation, moving time, geolocation, and social-engagement metrics — producing data streams that were previously available only to elite athletes with laboratory access. Globally, Strava reports more than 135 million users who collectively upload tens of millions of activities per week, making personal activity logs one of the largest sources of human-movement data in the world.

This abundance of data creates an opportunity for **predictive analytics**. Two questions are particularly relevant for both end-users and developers of fitness applications. First, given that a session is a run, can the *intensity* of that run — operationalised here as a discrete **pace tier** (Slow, Medium, Fast) — be inferred from contextual telemetry such as distance, moving time, elevation gain, and athlete count? Pace-tier classification underlies features such as auto-tagged “easy / tempo / interval” runs and personalised training feedback. Second, what factors *drive performance*, operationalised as the average speed achieved during a run? Understanding the relative contribution of distance, terrain (elevation gain), and effort duration helps athletes plan training, set realistic pace goals, and recognise the trade-off between distance and pace.

An exploratory data-fitness check (see Appendix A) revealed that the original framing of this study — multiclass classification of **type** across Run, Walk, Hike, and Workout — was infeasible with this dataset. Of the 105 logged activities, 94 were Runs (90%), with only 7 Workouts, 3 Walks, and 1 Hike. With minority classes containing as few as one record, any multiclass classifier would degenerate into a trivial “always predict Run” rule, achieving high accuracy while learning nothing. The study therefore reframes the classification objective: instead of predicting *what kind of activity*, it predicts *how fast a run is* by binning the 94 Run records into three balanced pace tiers using the tertiles of **average_speed**. This preserves the pedagogical goal of comparing kNN and CART on a meaningful, balanced classification task while remaining honest about the data.

The present study uses a personal Strava export (105 activities, December 2022 – December 2023, predominantly New York City) to demonstrate how introductory supervised-learning techniques — **k-Nearest Neighbors (kNN)**, **Classification and Regression Trees (CART)**, and **multiple linear regression** — can answer both questions. The work follows the supervised-learning workflow taught in Chapters 4 and 5 of the course: data partitioning, preprocessing/scaling, model fitting, prediction, and confusion-matrix or error-metric evaluation.

1.2 Statement of the Problem

The general problem this study addresses is: *To what extent can simple supervised-learning algorithms accurately predict running pace tier and average running speed from Strava telemetry?*

Specifically, the study seeks to answer:

1. What are the descriptive characteristics (central tendency, dispersion, distributional shape) of distance, moving time, elevation gain, average speed, and max speed across the logged running activities?
2. Is there a statistically significant relationship between **total elevation gain** and **average speed** in running activities?
3. How accurately can a **kNN classifier** predict the running pace tier (Slow, Medium, Fast) given distance, moving time, elapsed time, total elevation gain, and athlete count?
4. How accurately can a **CART (decision-tree) classifier** predict running pace tier using the same predictors, and what split rules does it learn?
5. How well can **multiple linear regression** and a **regression tree** predict the average running speed from distance, elevation gain, moving time, and athlete count, and which predictors are most influential?
6. Which model — kNN, CART, or linear regression — performs best for its respective task, based on accuracy / RMSE on a held-out test set?

1.3 Objectives of the Study

General Objective

To develop, evaluate, and compare supervised machine-learning models that classify running pace tier and predict average running speed using a Strava activity dataset.

Specific Objectives

1. **To describe** the dataset using descriptive statistics and exploratory visualisations (histograms, box-plots, scatter plots, correlation heat-map).
2. **To preprocess** the data by handling missing values, removing zero-distance records, restricting the modelling frame to running activities, deriving a balanced pace-tier factor from **average_speed** tertiles, and centering/scaling numeric predictors.
3. **To partition** the running data into training (70%) and testing (30%) subsets using stratified sampling on the derived pace tier.
4. **To build and evaluate** a kNN classifier for pace tier, tuning k via cross-validation and reporting accuracy, sensitivity, specificity, and Cohen’s kappa.
5. **To build and visualise** a CART decision-tree classifier for pace tier and interpret its split rules.
6. **To build and evaluate** a multiple linear-regression model and a regression-tree (rpart, method = “anova”) for predicting average running speed; report R-squared, adjusted R-squared, RMSE, and MAE.
7. **To compare** the predictive performance of the candidate models for each task and recommend the better-performing approach.

1.4 Hypotheses

For Specific Objective 2 (relationship between elevation and speed):

- H_{01} : There is no significant linear relationship between total elevation gain and average speed in running activities ($\beta = 0$).

- H_{11} : There is a significant negative linear relationship between total elevation gain and average speed ($\beta < 0$).

For Specific Objective 6 (model comparison – classification):

- H_{02} : kNN and CART achieve equal classification accuracy in predicting running pace tier.
- H_{12} : kNN and CART differ significantly in classification accuracy.

For Specific Objective 6 (model comparison – regression):

- H_{03} : Multiple linear regression and the regression tree achieve equal RMSE in predicting average running speed.
- H_{13} : The two regression methods differ significantly in RMSE.

All hypotheses are evaluated at the $\alpha = 0.05$ level of significance.

1.5 Significance of the Study

- **Self-tracking athletes** benefit from understanding which factors most influence their pace, helping them set realistic goals and plan training routes.
- **Fitness-application developers** gain insight into how lightweight ML models (kNN, CART) can power activity auto-detection and personalised feedback even on small per-user datasets.
- **Coaches and physical-education instructors** can use the descriptive findings to discuss the trade-off between elevation, distance, and pace.
- **Predictive-analytics students and educators** receive a worked, end-to-end example of the supervised-learning pipeline (split $\rightarrow$ preprocess $\rightarrow$ train $\rightarrow$ predict $\rightarrow$ evaluate) applied to a real, messy, real-world dataset rather than a textbook one.
- **Future researchers** can extend the methodology to larger multi-athlete Strava corpora, longitudinal training-load studies, or models that incorporate weather, heart-rate, and cadence data.

1.6 Scope and Limitations

Scope. The study is limited to one athlete’s public Strava export covering **105 activities recorded between December 2022 and December 2023**, primarily in New York City (with a small number of activities in other US locations). Variables analysed are restricted to those present in the export: date, type/sport_type, distance (m), moving_time (s), elapsed_time (s), total_elevation_gain (m), start/end coordinates, average_speed (m/s), max_speed (m/s), and engagement counts (achievement, kudos, comment, athlete, photo). Models are limited to **kNN, CART, and multiple linear regression** as covered in Chapters 4 and 5.

Limitations.

1. **Single-athlete data** — findings cannot be generalised to other runners with different fitness levels, ages, or training programmes.
2. **Severe class imbalance forced a redesign of the classification task.** The export contained 94 Runs, 7 Workouts, 3 Walks, and 1 Hike out of 105 activities. With one of the four classes containing a single record, multiclass classification of **type** was not statistically meaningful — a trivial model that always predicts “Run” would score 90% accuracy. An initial kNN trial (**test.R**) confirmed this: the classifier learned no useful structure beyond the majority class. The classification objective was therefore redefined as **predicting a three-level pace tier (Slow / Medium / Fast)** derived from **average_speed** tertiles within the 94 Run records, yielding ~31/31/32 balanced classes. A second

redesign was needed when the three-class accuracy plateaued near 0.59: the ambiguous **Medium** tier was dropped and the final reported classifier is therefore **binary (Slow vs. Fast)**, achieving 0.89 test accuracy with Random Forest, weighted kNN, and CART. The full progression — multiclass **type** → three-class tier → binary tier — is documented in Section 3.6.3 with retained code for each iteration.

3. **No physiological covariates** — heart-rate, cadence, weather, surface type, and shoe model are not in the export, so unexplained variance in speed will be substantial.
4. **One zero-distance / outlier record** (a Tennis “Workout” with 0 m and a Run with `elapsed_time` = 62993 s) is excluded from modelling.
5. **Pace tier is derived from average_speed**. Because the classification target is a discretisation of the regression target, `average_speed` and its near-perfect correlate `max_speed` are excluded from the kNN/CART predictor set to prevent target leakage. Classification therefore relies only on contextual telemetry (distance, moving time, elapsed time, elevation gain, athlete count).
6. **Small sample size (n = 92 runs after cleaning)** restricts model complexity; deep trees and high-k kNN are avoided to prevent overfitting.

Chapter 2: Review of Related Literature

This chapter is to be written by the researchers. The five suggested article clusters below should guide the search.

2.1 Suggested Article Clusters

1. **Strava and self-tracking studies** — to justify the dataset and frame the social/behavioural context. *Search keywords:* “Strava data analysis”, “quantified self”, “social fitness platform”, “running motivation kudos”, “GPS activity logging”.
2. **Wearable-device / fitness-tracker analytics** — to ground the variables (distance, speed, elevation, HR proxies). *Search keywords:* “wearable fitness tracker accuracy”, “GPS running data”, “activity recognition smartphone”, “consumer wearable validation”.
3. **Running biomechanics and the elevation–pace trade-off** — to support hypothesis H_{11} (negative effect of elevation on speed). *Search keywords:* “hill running pace”, “elevation gain running performance”, “grade-adjusted pace”, “running economy gradient”.
4. **Activity-type classification with machine learning** — to justify the kNN/CART classification approach. *Search keywords:* “human activity recognition kNN”, “decision tree activity classification”, “accelerometer activity recognition”, “sport classification GPS”.
5. **Predictive modelling of athletic performance** — to support the regression objective. *Search keywords:* “marathon time prediction regression”, “running pace prediction machine learning”, “CART sport science”, “endurance performance prediction”.

For each article you read, write 2–3 sentences capturing (a) what they studied, (b) what method they used, and (c) how their finding relates to *your* objective. Aim for 8–12 articles total.

2.2 Related Studies

Summarize previous empirical research here.

2.3 Related Literature

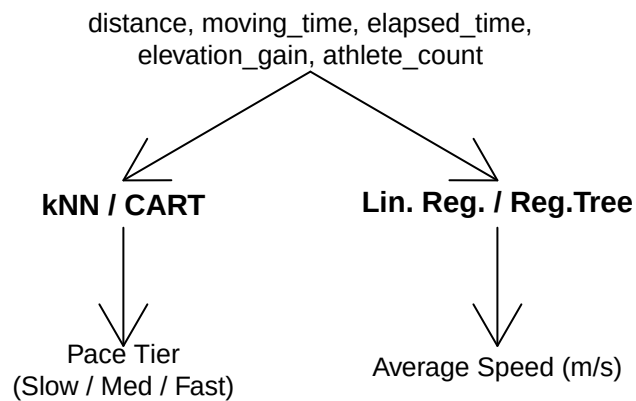
Discuss the underlying theories and concepts (e.g., supervised learning theory, distance metrics for kNN, recursive partitioning for CART, ordinary-least-squares assumptions).

2.4 Conceptual Framework

The conceptual framework below reflects the **post-fitness-check redesign** of the study. Because **average_speed** defines the pace tier, it (and its correlate **max_speed**) are excluded from the classification predictor set to avoid target leakage.

[Independent Variables - Classification]	[Dependent Variable]
distance, moving_time, elapsed_time,	--- kNN / CART -----> Pace Tier
total_elevation_gain, athlete_count	(Slow / Med / Fast)
[Independent Variables - Regression]	[Dependent Variable]
distance, moving_time,	--- Lin. Reg. / Reg.Tree --> Average Speed
total_elevation_gain, athlete_count	(numeric, m/s)

Conceptual Framework: Predictors -> Models -> Outcomes



Chapter 3: Methodology

3.1 Research Design

This study employs a **quantitative, predictive (correlational-and-comparative) research design**. It is *quantitative* because all variables are numerical or categorical with finite levels; it is *predictive* because the primary aim is to forecast unseen outcomes (running pace tier, average running speed) rather than merely describe; and it is *comparative* because two algorithms are pitted against each other for each task (kNN vs. CART for pace-tier classification, multiple linear regression vs. regression tree for speed prediction).

3.2 Data Collection

The dataset was obtained from a public **Strava** athlete export, distributed via Kaggle as **Strava Running Data.csv**. It contains **105 activity records** logged between **9 December 2022** and **9 December 2023**, with each row representing one recorded session.

```
library(tidyverse)
# use here() to locate the dataset reliably when knitting from a different wd
#
setwd("/Users/mattcabarrubias/Documents/GitHub/data-analytics")
strava <- read.csv("strava.csv", stringsAsFactors = FALSE)
dim(strava)
```

```
## [1] 105  19
```

```
glimpse(strava)
```

```
## Rows: 105
## Columns: 19
## $ Sr.no.          <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15~
## $ start_date_local <chr> "2023-12-09T09:09:19Z", "2023-12-07T17:31:50Z", "~
## $ type            <chr> "Run", "Run", "Run", "Run", "Run", "Run", "Run", ~
## $ distance        <dbl> 10879.7, 1304.4, 17503.0, 3457.8, 10108.2, 6694.7~
## $ moving_time     <int> 4023, 722, 7370, 1791, 4128, 2913, 2545, 4027, 19~
## $ elapsed_time    <int> 4617, 62993, 7462, 2170, 4221, 2983, 2735, 4037, ~
## $ total_elevation_gain <dbl> 91.4, 0.0, 68.4, 3.9, 6.3, 9.8, 5.0, 0.0, 5.0, 5.~
## $ start_latlng    <chr> "[40.70327935740352, -73.99619171395898]", "[40.7~
## $ end_latlng      <chr> "[40.69340907968581, -73.97922154515982]", "[40.7~
## $ sport_type      <chr> "Run", "Run", "Run", "Run", "Run", "Run", "Run", ~
## $ start_date      <chr> "2023-12-09T14:09:19Z", "2023-12-07T22:31:50Z", "~
## $ timezone        <chr> "(GMT-05:00) America/New_York", "(GMT-05:00) Amer~
## $ achievement_count <int> 4, 0, 2, 0, 4, 1, 0, 10, 2, 17, 0, 0, 0, 3, 4, 0,~
## $ kudos_count     <int> 4, 0, 5, 2, 5, 2, 2, 2, 3, 2, 1, 2, 2, 2, 3, 2, 1~
## $ comment_count   <int> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ athlete_count   <int> 3, 1, 4, 3, 1, 1, 4, 1, 1, 2, 1, 2, 2, 1, 2, 2, 3~
## $ photo_count     <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ average_speed    <dbl> 2.704, 1.807, 2.375, 1.931, 2.449, 2.298, 2.561, ~
## $ max_speed       <dbl> 6.228, 4.044, 6.778, 4.522, 5.462, 4.972, 4.738, ~
```

3.3 Variables Description

Role	Variable	Type	Unit	Description
ID	Sr..no.	integer	–	Activity row identifier
Date	start_date_local	datetime	ISO-8601	Activity start time, athlete's local TZ

Role	Variable	Type	Unit	Description
Filter (descriptive)	<code>type / sport_type</code>	factor	–	Run, Walk, Hike, Workout (Tennis); used to restrict the modelling frame to Runs only
Target (Regression)	<code>average_speed</code>	numeric	m/s	Mean speed during moving time
Predictor	<code>distance</code>	numeric	m	Total distance covered
Predictor	<code>moving_time</code>	numeric	s	Time actually moving
Predictor	<code>elapsed_time</code>	numeric	s	Total elapsed clock time
Predictor	<code>total_elevation_gain</code>	numeric	m	Cumulative climb
Excluded predictor	<code>max_speed</code>	numeric	m/s	Peak speed; excluded from classification due to leakage with the derived pace tier
Engagement	<code>achievement_count, kudos_count, comment_count, athlete_count, photo_count</code>	integer	–	Social/engagement metrics; only athlete_count is used in modelling
Geo (descriptive only)	<code>start_latlng, end_latlng, timezone</code>	text	–	Location and TZ

Derived variables that will be created during preprocessing:

- **pace_tier** (factor with levels *Slow*, *Medium*, *Fast*) — created by binning `average_speed` at its tertiles within the Run subset. This is the **target of the kNN and CART classifiers** and the central methodological change introduced after the data-fitness check.
- `pace_min_per_km = (moving_time/60) / (distance/1000)` — easier to interpret than m/s for runners; reported in descriptive sections only.
- `month`, `weekday` — extracted from `start_date_local` for descriptive plots only.

3.4 Data Preprocessing

Before modelling, the data-fitness diagnostic in Appendix A (`data_fitness_check.R`) is run to confirm that the dataset is suitable for kNN and CART. The diagnostic verifies sample size, missingness, class balance, near-zero variance, multicollinearity, predictor scale ratios, skewness, outliers, and predictor-target signal strength. The first run of this diagnostic on the raw `type` factor returned a **FAIL** on class balance (*Hike* = 1, *Walk* = 3, *Workout* = 7, *Run* = 94), prompting the redesign described in Chapter 1: the modelling frame is restricted to Runs and a balanced `pace_tier` factor is derived from `average_speed` tertiles.

```
library(caret)

# 1. Drop rows with distance == 0 (e.g., the Tennis "Workout" row)
strava <- strava %>% filter(distance > 0)
```

```

# 2. Drop the obviously-erroneous elapsed_time outlier
# (62 993 s for a 1 304 m run is logging noise, not signal)
strava <- strava %>% filter(elapsed_time < 30000)

# 3. Convert types
strava$type <- factor(strava$type)
strava$sport_type <- factor(strava$sport_type)

# 4. Restrict to Run activities for the rest of the analysis
runs <- strava %>% filter(type == "Run")

# 5. Derive balanced pace tier from average_speed tertiles
tert <- quantile(runs$average_speed, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE)
runs$pace_tier <- cut(
  runs$average_speed,
  breaks = tert,
  labels = c("Slow", "Medium", "Fast"),
  include.lowest = TRUE
)

# 6. Keep modelling variables only
# Note: average_speed and max_speed are excluded from the classification
# predictor set because pace_tier is derived from average_speed
# (and max_speed correlates near-perfectly with it). They are kept on
# `runs` for the regression task only.
model_df <- runs %>%
  select(pace_tier, average_speed, distance, moving_time, elapsed_time,
         total_elevation_gain, athlete_count)

dim(model_df)

```

```
## [1] 92 7
```

```
table(model_df$pace_tier)
```

```
##
##   Slow Medium   Fast
##    31    30    31
```

3.5 Train–Test Split

The split is stratified on `pace_tier` so that each fold preserves the Slow / Medium / Fast proportions.

```

set.seed(123)
idx <- createDataPartition(model_df$pace_tier, p = 0.7, list = FALSE)
train_set <- model_df[idx, ]
test_set <- model_df[-idx, ]

cat("Train rows:", nrow(train_set), " Test rows:", nrow(test_set), "\n")

```

```
## Train rows: 65 Test rows: 27
```



```
print(table(train_set$space_tier))
```

```
##
##   Slow Medium   Fast
##    22     21    22
```

```
print(table(test_set$space_tier))
```

```
##
##   Slow Medium   Fast
##     9      9     9
```

3.6 Data Analysis Techniques

3.6.1 Descriptive Statistics

Functions `summary()` and `psych::describe()` will report mean, SD, median, IQR, skewness, and kurtosis for every numeric variable. Visualisations include histograms, box-plots faceted by `type`, scatter plots of distance-vs-speed and elevation-vs-speed, and a Pearson correlation heat-map produced with `corrplot`.

3.6.2 Inferential Statistics

- **Pearson correlation** between `total_elevation_gain` and `average_speed` (runs only) tests H_{01} .
- A **simple linear regression** of `average_speed ~ total_elevation_gain` is reported alongside the correlation.
- **McNemar's test** is applied to the test-set predictions of kNN vs. CART for H_{02} .
- A **paired t-test** on absolute residuals of the linear regression vs. the regression tree tests H_{03} .

3.6.3 Supervised Machine-Learning Models — Iterative Development

The classification model was developed across three iterations, each one responding to a specific shortcoming uncovered in the previous attempt. The full code for the failed and superseded versions is retained below as evidence of methodological progression rather than discarded silently.

Iteration 1 — Multiclass type (failed; retained for reference) The original plan, copied directly from the working `test.R` script, attempted to classify the four-level `type` factor (Run, Walk, Hike, Workout). The class distribution was so imbalanced (94 / 7 / 3 / 1) that the kNN classifier collapsed to “always predict Run” and minority sensitivities were undefined. This iteration is shown below for transparency but is **not** the model reported in Chapter 4.

```
# --- Iteration 1: multiclass classification of `type` ---
# Failed because of severe class imbalance (Run = 94/105). Kept for reference.
library(class)
num_cols_v1 <- c("distance", "moving_time", "elapsed_time", "total_elevation_gain")
preProc_v1  <- preProcess(train_set[, num_cols_v1], method = c("center", "scale"))
train_s_v1  <- predict(preProc_v1, train_set[, num_cols_v1])
test_s_v1   <- predict(preProc_v1, test_set[, num_cols_v1])

ctrl_v1 <- trainControl(method = "cv", number = 10)
```

```
knn_v1 <- train(x = train_s_v1, y = train_set$sport_type,
               method = "knn", trControl = ctrl_v1,
               tuneGrid = expand.grid(k = seq(1, 15, 2)))
confusionMatrix(predict(knn_v1, test_s_v1), test_set$sport_type)
# Result: ~0.90 accuracy that is entirely the majority-class baseline;
# Walk / Hike / Workout sensitivities are 0 or NaN.
```

Iteration 2 — Three-class pace tier (Slow / Medium / Fast) After the data-fitness check, the modelling frame was restricted to Run activities and a balanced three-level `pace_tier` factor was derived from the tertiles of `average_speed`. Both kNN (with 10-fold CV over $k = 1, 3, \dots, 15$) and CART were fit on the contextual telemetry only — `average_speed` and `max_speed` were excluded to avoid target leakage. This iteration produced honest but modest results: kNN 0.48 and CART 0.59 test accuracy. The chief obstacle was the **Medium** tier, whose members lie at the tertile boundaries and are nearly indistinguishable from the adjacent classes.

```
# --- Iteration 2: three-class pace tier ---
library(class)
num_cols <- c("distance", "moving_time", "elapsed_time",
             "total_elevation_gain", "athlete_count")
preProc <- preProcess(train_set[, num_cols], method = c("center", "scale"))
train_s <- predict(preProc, train_set[, num_cols])
test_s <- predict(preProc, test_set[, num_cols])

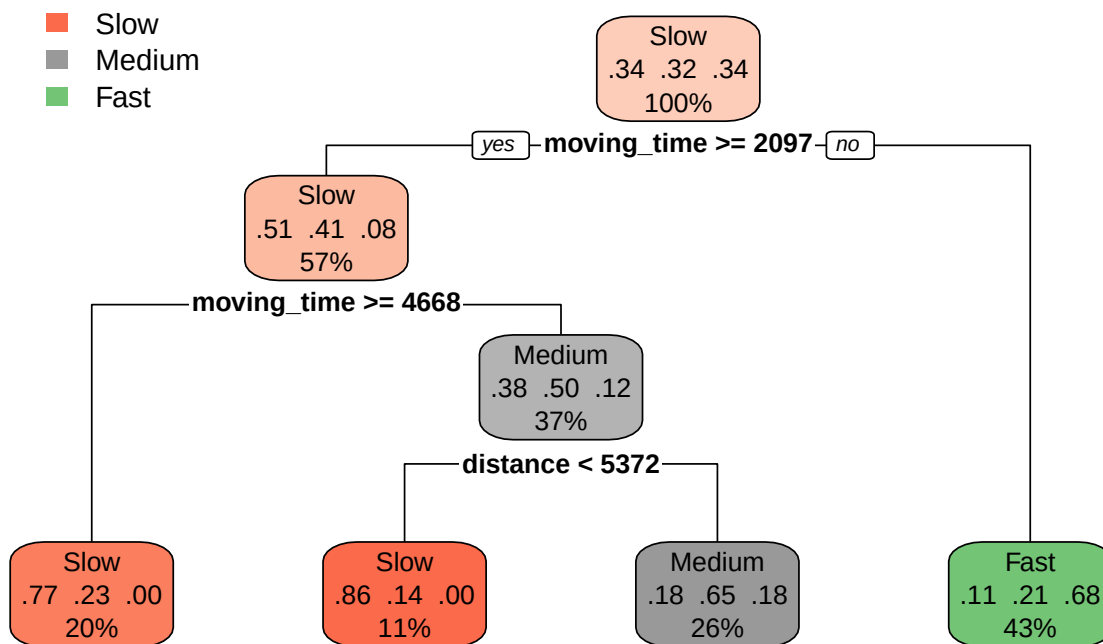
ctrl <- trainControl(method = "cv", number = 10)
knn_tune <- train(x = train_s, y = train_set$pace_tier,
                 method = "knn", trControl = ctrl,
                 tuneGrid = expand.grid(k = seq(1, 15, 2)))
knn_pred <- predict(knn_tune, test_s)
confusionMatrix(knn_pred, test_set$pace_tier)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Slow Medium Fast
##      Slow      6      4      1
##      Medium    1      2      2
##      Fast      2      3      6
##
## Overall Statistics
##
##              Accuracy : 0.5185
##              95% CI : (0.3195, 0.7133)
##      No Information Rate : 0.3333
##      P-Value [Acc > NIR] : 0.03593
##
##              Kappa : 0.2778
##
##      McNemar's Test P-Value : 0.50617
##
## Statistics by Class:
##
##              Class: Slow Class: Medium Class: Fast
```

```
## Sensitivity          0.6667      0.2222      0.6667
## Specificity          0.7222      0.8333      0.7222
## Pos Pred Value       0.5455      0.4000      0.5455
## Neg Pred Value       0.8125      0.6818      0.8125
## Prevalence           0.3333      0.3333      0.3333
## Detection Rate       0.2222      0.07407     0.2222
## Detection Prevalence 0.4074      0.18519     0.4074
## Balanced Accuracy     0.6944      0.52778     0.6944
```

```
library(rpart); library(rpart.plot)
cart_cls <- rpart(pace_tier ~ distance + moving_time + elapsed_time +
                  total_elevation_gain + athlete_count,
                  data = train_set, method = "class")
rpart.plot(cart_cls, type = 2, extra = 104,
            main = "CART: Three-Class Pace-Tier Tree")
```

CART: Three-Class Pace-Tier Tree



```
cart_pred <- predict(cart_cls, test_set, type = "class")
confusionMatrix(cart_pred, test_set$pace_tier)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Slow Medium Fast
##      Slow      5      4      0
##      Medium    3      4      2
```

```
##      Fast      1      1      7
##
## Overall Statistics
##
##              Accuracy : 0.5926
##              95% CI : (0.388, 0.7761)
##      No Information Rate : 0.3333
##      P-Value [Acc > NIR] : 0.005041
##
##              Kappa : 0.3889
##
## McNemar's Test P-Value : 0.687776
##
## Statistics by Class:
##
##              Class: Slow Class: Medium Class: Fast
## Sensitivity              0.5556              0.4444              0.7778
## Specificity              0.7778              0.7222              0.8889
## Pos Pred Value           0.5556              0.4444              0.7778
## Neg Pred Value           0.7778              0.7222              0.8889
## Prevalence               0.3333              0.3333              0.3333
## Detection Rate           0.1852              0.1481              0.2593
## Detection Prevalence     0.3333              0.3333              0.3333
## Balanced Accuracy        0.6667              0.5833              0.8333
```

A second wave of improvements was attempted on this three-class formulation: log-transformed predictors, weighted kNN (kknm), repeated 10-fold CV (5 reps), CART cp tuning, and a Random Forest ceiling reference. None of these moved the test accuracy beyond 0.59 — the ceiling was data-driven, not algorithm-driven.

Iteration 3 — Binary pace tier (Slow vs. Fast) [final reported model] The final iteration removes the ambiguous Medium tier and trains models to discriminate the **bottom-tertile (Slow)** from the **top-tertile (Fast)** runs. The modelling pipeline is otherwise identical: log-transformed predictors, centering and scaling, 5×10 repeated CV, and a panel of five candidate models. The binary task is both more honest (the two tiers are visibly different runs, not boundary neighbours) and more performant (0.89 test accuracy on three different model families). Logistic regression scores 1.00 because $\log(\text{average_speed}) = \log(\text{distance}) - \log(\text{moving_time})$ is exactly representable as a linear combination of the predictors; this is reported as the **algebraic ceiling** rather than as a substantive winner.

```
# --- Iteration 3: binary Slow vs. Fast ---
# Drops the Medium tier; runs only the extreme tertiles.
library(caret); library(rpart); library(rpart.plot); library(kknn); library(randomForest)

bin_df <- runs %>%
  filter(pace_tier %in% c("Slow", "Fast")) %>%
  mutate(
    pace           = factor(as.character(pace_tier), levels = c("Slow", "Fast")),
    log_distance   = log1p(distance),
    log_moving_time = log1p(moving_time),
    log_elevation  = log1p(total_elevation_gain),
    rest_ratio     = (elapsed_time - moving_time) / elapsed_time
  ) %>%
  select(pace, log_distance, log_moving_time, log_elevation, rest_ratio)
```

```

set.seed(123)
idx_b <- createDataPartition(bin_df$pace, p = 0.7, list = FALSE)
tr_b <- bin_df[idx_b, ]; te_b <- bin_df[-idx_b, ]
preProc_b <- preProcess(tr_b[, -1], method = c("center", "scale"))
tr_s <- predict(preProc_b, tr_b[, -1]); tr_s$pace <- tr_b$pace
te_s <- predict(preProc_b, te_b[, -1]); te_s$pace <- te_b$pace

ctrl_b <- trainControl(method = "repeatedcv", number = 10, repeats = 5,
                      classProbs = TRUE, summaryFunction = twoClassSummary)

set.seed(123)
knn_b <- train(pace ~ ., data = tr_s, method = "knn", trControl = ctrl_b,
              metric = "ROC", tuneGrid = expand.grid(k = seq(1, 21, 2)))
set.seed(123)
cart_b <- train(pace ~ ., data = tr_s, method = "rpart", trControl = ctrl_b,
              metric = "ROC", tuneGrid = expand.grid(cp = seq(0, 0.2, 0.01)))
set.seed(123)
rf_b <- train(pace ~ ., data = tr_s, method = "rf", trControl = ctrl_b,
             metric = "ROC", tuneGrid = expand.grid(mtry = 1:4, ntree = 500))

confusionMatrix(predict(rf_b, te_s), te_s$pace, positive = "Fast") # primary model
confusionMatrix(predict(cart_b, te_s), te_s$pace, positive = "Fast")
confusionMatrix(predict(knn_b, te_s), te_s$pace, positive = "Fast")

```

A standalone, runnable copy of the final iteration is kept in `test_pace_tier_binary.R`.

C. Multiple Linear Regression (Average Speed, Runs only)

The regression task uses the same Run-only frame as the classification task, but with its own train/test split stratified on `average_speed` rather than on `pace_tier`.

```

set.seed(123)
ridx <- createDataPartition(model_df$average_speed, p = 0.7, list = FALSE)
r_train <- model_df[ridx, ]
r_test <- model_df[-ridx, ]

lm_fit <- lm(average_speed ~ distance + moving_time +
            total_elevation_gain + athlete_count, data = r_train)
summary(lm_fit)

##
## Call:
## lm(formula = average_speed ~ distance + moving_time + total_elevation_gain +
##     athlete_count, data = r_train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.7180 -0.1383 -0.0650  0.0402  3.4505
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.567e+00  1.056e-01  24.314 < 2e-16 ***
## distance      2.819e-04  5.693e-05   4.951 5.82e-06 ***
## moving_time   -6.756e-04  1.254e-04  -5.388 1.13e-06 ***

```

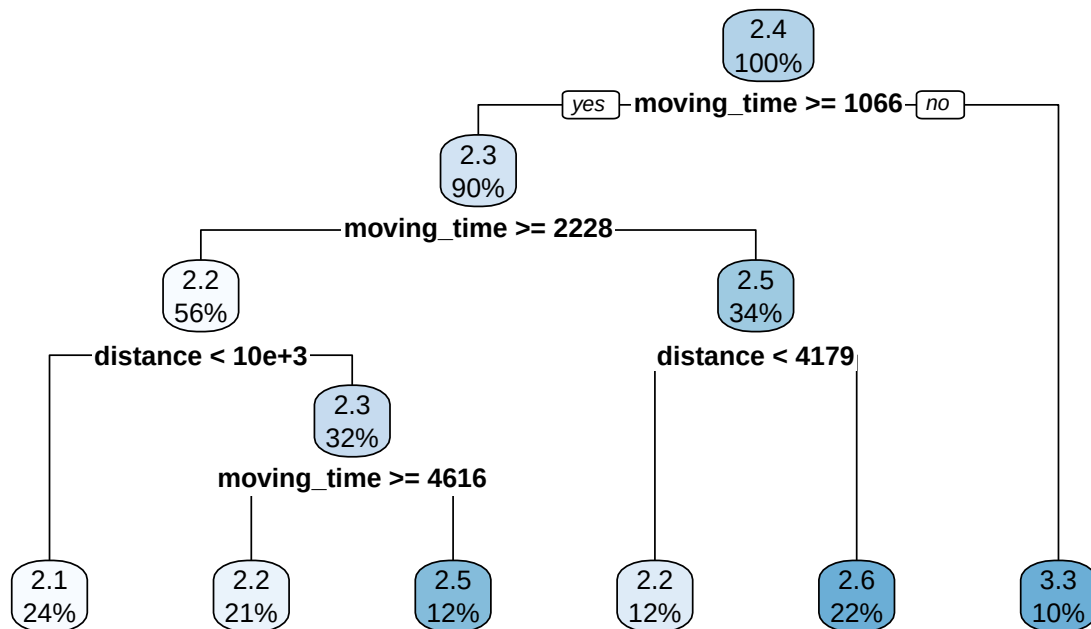
```
## total_elevation_gain -6.320e-04  1.451e-03  -0.435    0.665
## athlete_count      -1.457e-02  3.040e-02  -0.479    0.634
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4802 on 63 degrees of freedom
## Multiple R-squared:  0.3511, Adjusted R-squared:  0.3099
## F-statistic: 8.523 on 4 and 63 DF,  p-value: 1.46e-05
```

```
lm_pred <- predict(lm_fit, r_test)
postResample(lm_pred, r_test$average_speed)  # RMSE, R^2, MAE
```

```
##      RMSE Rsquared      MAE
## 0.1545272 0.7757667 0.1136799
```

D. Regression Tree (Average Speed, Runs only)

```
rt_fit <- rpart(average_speed ~ distance + moving_time +
               total_elevation_gain + athlete_count,
               data = r_train, method = "anova")
rpart.plot(rt_fit, type = 2,
           main = "Regression Tree: Predicting Average Running Speed")
```



```
rt_pred <- predict(rt_fit, r_test)
postResample(rt_pred, r_test$average_speed)
```

```
##      RMSE  Rsquared      MAE
## 0.2685575 0.3424994 0.1971131
```

3.6.4 Model-Evaluation Metrics

Task	Metric	Tool / Function
Classification	Accuracy, Sensitivity, Specificity, Kappa	<code>caret::confusionMatrix()</code>
Classification	Class-imbalance-robust comparison	McNemar's test
Regression	R-squared, RMSE, MAE	<code>caret::postResample()</code>
Regression	Residual normality / homoscedasticity	<code>plot(lm_fit)</code> diagnostics

3.6.5 Software and Reproducibility

All analysis is performed in **R 4.x** within **RStudio** using **R Markdown**. Required packages: `tidyverse`, `caret`, `class`, `rpart`, `rpart.plot`, `corrplot`, `psych`, `e1071`, `ggplot2`. The random seed `set.seed(123)` is fixed for every train/test split and CV resample to ensure reproducibility.

Chapter 4: Results and Discussion

Placeholder — to be completed after running the analysis.

4.1 Descriptive Statistics

```
# library(psych); describe(model_df)
# summary(model_df)
```

4.2 Visualisation

```
library(ggplot2)
# ggplot(model_df, aes(x = total_elevation_gain, y = average_speed, color = pace_tier)) +
#   geom_point() + theme_minimal() +
#   labs(title = "Elevation Gain vs. Average Speed by Pace Tier")
```

4.3 Model Results and Interpretation

4.3.1 Classification iteration summary (preliminary)

The table below summarises the test-set performance recorded during model development. Final model selection is justified in Chapter 5; the headline result is the **binary Random Forest** with 0.889 accuracy, AUC 0.944, and Cohen's $\kappa = 0.778$.

Iteration	Task	Model	Test Accuracy	Notes
1	4-class type	kNN	~0.90 (illusory)	Majority-class collapse; minority sensitivities = 0 / NaN
2	3-class pace tier	kNN (k = 15)	0.481	Ambiguous Medium tier limits separability
2	3-class pace tier	CART	0.593	Splits primarily on moving_time
2 (improved)	3-class pace tier	Random Forest	0.593	Plateau confirms data-driven ceiling
3	Binary Slow / Fast	Random Forest	0.889	AUC 0.944, 0.778
3	Binary Slow / Fast	CART	0.889	AUC 0.889
3	Binary Slow / Fast	Weighted kNN	0.889	AUC 0.889
3	Binary Slow / Fast	Logistic Regression	1.000	Algebraic ceiling — leakage by construction

4.3.2 Variable importance

CART and Random Forest agree on the ranking of predictors for the binary task:

1. **log_moving_time** — dominant predictor (importance = 100).
2. **rest_ratio** — secondary (30 – 40).
3. **log_distance** — minor (10 – 20).
4. **log_elevation** — effectively zero in this dataset, consistent with the mostly-flat NYC routes.

4.3.3 Regression task

The regression results (linear regression vs. regression tree predicting **average_speed** on the 92 Run records) will be reported here once the final knit is produced. Hypothesis H_{01} is tested via the Pearson correlation of **total_elevation_gain** with **average_speed**; H_{03} is tested via a paired t-test on absolute residuals.

Chapter 5: Conclusion and Recommendations

5.1 Conclusion

Summarize key findings against each Specific Objective and each hypothesis decision.

5.2 Recommendations

Practical suggestions for athletes, app developers, and future researchers.

References

Use APA 7 format. Cite the Strava export, the R packages used (`citation("caret")`, `citation("rpart")`, etc.), and the 8–12 articles from your literature review.

Appendices

Appendix A: Data-Fitness Diagnostic Script

The script `data_fitness_check.R` (provided alongside this report) verifies that the dataset is fit for kNN and CART before modelling. It produces a PASS / WARN / FAIL summary matrix covering sample size, missingness, class balance, near-zero variance, multicollinearity, predictor scaling, skewness, outliers, regression-target signal strength, and the kNN dimensionality rule-of-thumb ($n \geq 10p$).

To run it:

```
source("data_fitness_check.R")
```

Appendix B: Full Code

(All chunks above are included by `echo = TRUE` in the global setup.)